



Connect a GitHub Repo with AWS

CH

Arpita Chowdhury

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[ec2-user@ip-172-31-24-63 ~]$ sudo dnf update -y
sudo dnf install git -y
(6/8): perl-TermReadKey-2.38-9.amzn2023.0.2.x86_64.rpm 1.1 MB/s | 36 kB 00:00
(7/8): perl-lib-0.65-477.amzn2023.0.7.x86_64.rpm 584 kB/s | 15 kB 00:00
(8/8): git-core-2.50.1-1.amzn2023.0.1.x86_64.rpm 27 MB/s | 4.9 MB 00:00

Total 34 MB/s | 7.9 MB 00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing :
  Installing : git-core-2.50.1-1.amzn2023.0.1.x86_64 1/1
  Installing : git-core-doc-2.50.1-1.amzn2023.0.1.noarch 1/8
  Installing : perl-lib-0.65-477.amzn2023.0.7.x86_64 2/8
  Installing : perl-TermReadKey-2.38-9.amzn2023.0.2.x86_64 3/8
  Installing : perl-File-Find-1.37-477.amzn2023.0.7.noarch 4/8
  Installing : perl-Error-1.0.17029-5.amzn2023.0.2.noarch 5/8
  Installing : perl-Git-2.50.1-1.amzn2023.0.1.noarch 6/8
  Installing : git-2.50.1-1.amzn2023.0.1.x86_64 7/8
  Running scriptlet: git-2.50.1-1.amzn2023.0.1.x86_64 8/8
  Verifying : git-2.50.1-1.amzn2023.0.1.x86_64 1/8
  Verifying : git-core-2.50.1-1.amzn2023.0.1.x86_64 2/8
  Verifying : git-core-doc-2.50.1-1.amzn2023.0.1.noarch 3/8
  Verifying : perl-Error-1.0.17029-5.amzn2023.0.2.noarch 4/8
  Verifying : perl-File-Find-1.37-477.amzn2023.0.7.noarch 5/8
  Verifying : perl-Git-2.50.1-1.amzn2023.0.1.noarch 6/8
  Verifying : perl-TermReadKey-2.38-9.amzn2023.0.2.x86_64 7/8
  Verifying : perl-lib-0.65-477.amzn2023.0.7.x86_64 8/8

Installed:
git-2.50.1-1.amzn2023.0.1.x86_64 git-core-2.50.1-1.amzn2023.0.1.x86_64
git-core-doc-2.50.1-1.amzn2023.0.1.noarch perl-Error-1.0.17029-5.amzn2023.0.2.noarch
perl-File-Find-1.37-477.amzn2023.0.7.noarch perl-Git-2.50.1-1.amzn2023.0.1.noarch
perl-TermReadKey-2.38-9.amzn2023.0.2.x86_64 perl-lib-0.65-477.amzn2023.0.7.x86_64

Complete!
[ec2-user@ip-172-31-24-63 ~]$ git --version
git version 2.50.1
[ec2-user@ip-172-31-24-63 ~]$
```



Introducing Today's Project!

In this project, I will demonstrate setting up Git and GitHub, connecting my existing web application to a Git repository, making code changes, and tracking those updates in GitHub with proper documentation. I'm doing this project to learn how source control works in real-world DevOps workflows and why Git repositories are essential for collaboration, version tracking, and building an automated CI/CD pipeline.

Key tools and concepts

Services I used were Amazon EC2, GitHub, and Visual Studio Code, along with Git for version control and Apache Maven for building the Java web application. Key concepts I learnt include launching and securing EC2 instances, remote development using SSH and VS Code, initializing and managing Git repositories, staging/committing/pushing code to GitHub, handling authentication with Personal Access Tokens, using .gitignore to exclude unwanted files, and documenting projects clearly with Markdown for portfolio readiness.

Project reflection

This project took me approximately 4 hours. The most challenging part was troubleshooting SSH connectivity issues, correctly configuring Git and GitHub authentication, and ensuring the repository was initialized in the right project directory. It was most rewarding to finally see my code successfully pushed to GitHub and updated in real time, confirming that my EC2, Git, and GitHub integration was working end to end.

I did this project because I wanted hands-on experience with real-world DevOps workflows—building a web application in the cloud, managing it with Git and GitHub, and preparing it for an automated CI/CD pipeline using AWS tools.



Arpita Chowdhury

NextWork Student

nextwork.org

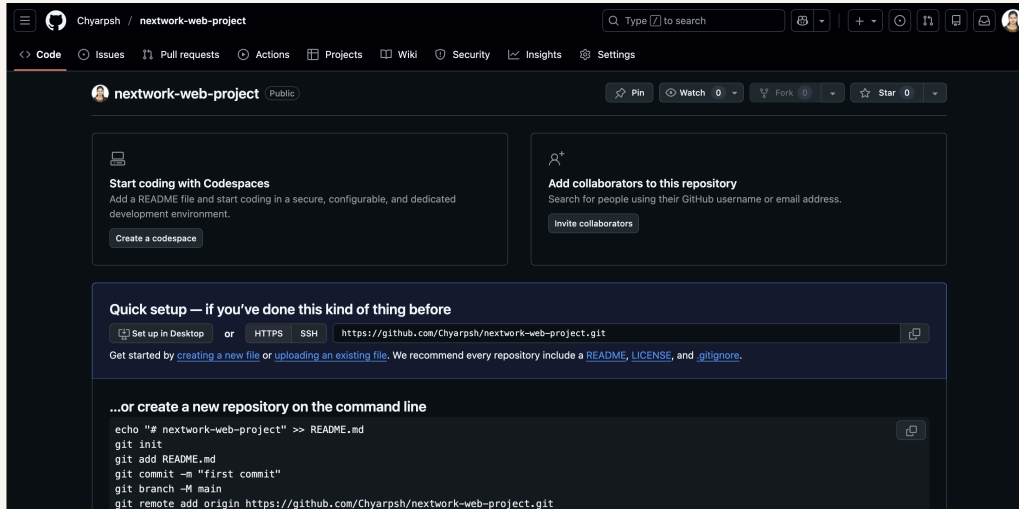
This project is part two of a series of DevOps projects where I'm building a CI/CD pipeline. I'll be working on the next project to automate the build and deployment process using AWS CI/CD services, further reducing manual steps and improving reliability.



Git and GitHub

Git is a distributed version control system that helps track changes in code, manage different versions of a project, and collaborate with others safely. It allows developers to see what changed, when it changed, and who made the change making it a core tool in CI/CD pipelines. I installed Git using the commands: `sudo dnf update -y` `sudo dnf install git -y`

GitHub is an online platform that hosts Git repositories, allowing developers to store, manage, and share their code while tracking every change made over time. I'm using GitHub in this project to securely store my web application's source code, track version history, and integrate it into a CI/CD pipeline as part of my DevOps learning journey.





My local repository

A Git repository is a version-controlled storage location for a project's files and code, where Git tracks changes, maintains history, and allows developers to manage and collaborate on different versions of a project.

`git init` is a command that creates a new local Git repository by enabling version control for a project folder. I ran `git init` inside my web application's project directory on the EC2 instance so Git could start tracking my project files.

A branch in Git is an independent line of development that allows you to work on changes without affecting the main codebase. After running `git init`, the response from the terminal was a message confirming that an empty Git repository was initialized and indicating the default branch (such as `main` or `master`).



```
● [ec2-user@ip-172-31-24-63 ~]$ git --version
git version 2.50.1
● [ec2-user@ip-172-31-24-63 ~]$ pwd
/home/ec2-user
● [ec2-user@ip-172-31-24-63 ~]$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/ec2-user/.git/
○ [ec2-user@ip-172-31-24-63 ~]$
```



To push local changes to GitHub, I ran three commands

`git add`

The first command I ran was `git remote add origin https://github.com/Chyarpsh/nextwork-web-project.git`, which linked my local Git repository on the EC2 instance to my GitHub repository. A staging area is an intermediate space where selected file changes are prepared and reviewed before being permanently saved in a commit to the Git repository.

`git commit`

The second command I ran was `git add .`, which staged all the modified and new files in my project so they were ready to be committed. Using `-m` means I can include a commit message directly in the command to describe the changes being saved in that commit.

`git push`

The third command I ran was `git commit -m "Updated index.jsp with new content"`, which saved my staged changes to the local Git repository as a new version. Using `-u` means Git sets the upstream branch, linking my local branch to the remote GitHub branch so future `git push` and `git pull` commands can run without specifying the branch name.



Authentication

When I commit changes to GitHub, Git asks for my credentials because it needs to authenticate me with GitHub to verify that I have permission to push changes to the remote repository and securely update the code stored there.

Local Git identity

Git needs my name and email because they are used to identify the author of each commit, allowing Git to track who made specific changes in the project's version history and maintain clear accountability and collaboration records.

Running git log showed me that my repository has one commit, which is the initial commit for the NextWork web project. It also showed who made the commit (the EC2 default user), when it was made, and that this commit is currently at the HEAD of the master branch and synced with origin/master, meaning my local repository and GitHub repository are up to date.



```
● [ec2-user@ip-172-31-24-63 nextwork-web-project]$ git log
commit b7221a032fe0b4cb67693f99d7e66fd9b41d163a (HEAD -> master, origin/master)
Author: EC2 Default User <ec2-user@ip-172-31-24-63.ec2.internal>
Date:   Sun Jan 4 04:30:45 2026 +0000

    Initial commit - NextWork web project
○ [ec2-user@ip-172-31-24-63 nextwork-web-project]$
```

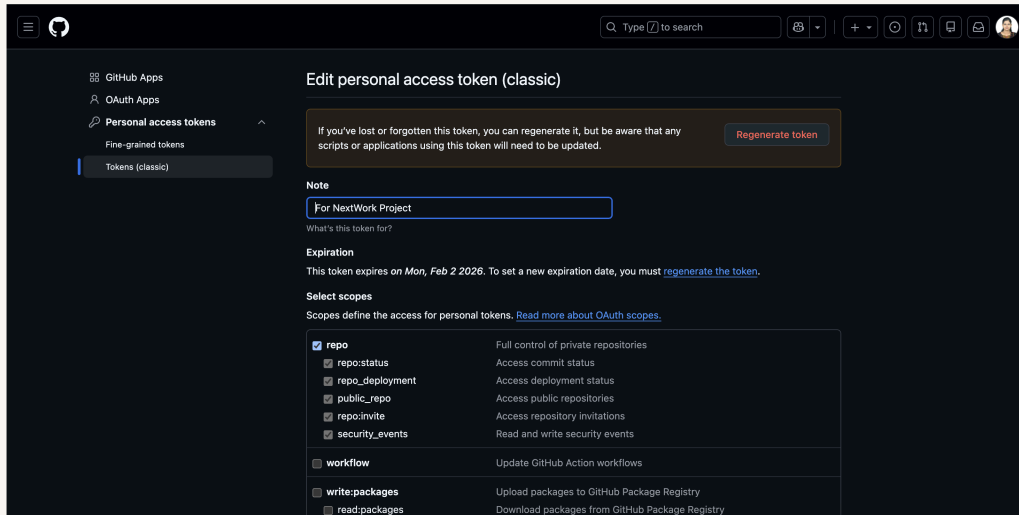


GitHub tokens

GitHub authentication failed when I entered my password because GitHub no longer supports password-based authentication for Git operations over HTTPS, so I needed to use a Personal Access Token (PAT) instead of my GitHub password.

A GitHub token is a secure, auto-generated credential that allows tools like Git to authenticate with GitHub without using a password. It acts like a temporary, permission-based key that grants access to specific actions, such as pushing code to a repository. I'm using one in this project because GitHub no longer allows password-based authentication for Git operations, so a Personal Access Token is required to securely push my code from an EC2 instance to my GitHub repository.

I could set up a GitHub token by going to GitHub Settings → Developer settings → Personal access tokens → Tokens (classic), generating a new token with the required permissions (such as repo), copying the token when it was created, and then using it as the password when authenticating Git commands from my EC2 instance.





Making changes again

I wanted to see Git working in action, so I edited index.jsp, committed the changes locally on my EC2 instance, and pushed the commit to my GitHub repository to verify that the updates appeared online. I couldn't see the changes in my GitHub repo initially because the updated file hadn't been committed and pushed yet. Git only syncs changes to GitHub after a successful git commit and git push.

I finally saw the changes in my GitHub repo after I ran git add, git commit, and git push, which uploaded my latest code changes to GitHub.

```
1 <html>
2
3 <body>
4   <h2>Hello <Arpita>! </h2>
5
6   <p>If you see this line in Github, that means your latest changes are getting pushed to your cloud repo :o</p>
7
8 </body>
9
10 </html>
```



Setting up a README file

As a finishing touch to my GitHub repository, I added a README file, which acts as the front page of the project and helps others understand what the repository contains. I added a README file by creating a README.md file and documenting my project goals, tools used, and what I learned during the process.

My README is written in Markdown because it allows me to clearly document my project in a way that is both developer-friendly and visually structured when viewed on GitHub. Markdown makes my repository easier for others (and recruiters) to understand at a glance. Special characters can help you format text in Markdown, such as `#` to create section headers, `*` or `-` to create lists, `**` to emphasize important text, and triple backticks to display commands and code snippets.

My README file has 6 sections that outline the project introduction, tools and technologies used, setup instructions, CI/CD goals, contact details, and a conclusion summarizing my learning and future plans.



Arpita Chowdhury

NextWork Student

nextwork.org

 Chyarpsh create README	4f11fb5 · now	 3 Commits
 src/main/webapp	Add new line to index.jsp	11 minutes ago
 .gitignore	Initial commit - NextWork web project	29 minutes ago
 README.md	create README	now
 pom.xml	Initial commit - NextWork web project	29 minutes ago



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

